

CS516: Parallelization of Programs

Runtime Parallelization Techniques

Vishwesh Jatala

Assistant Professor

Department of CSE

Indian Institute of Technology Bhilai

vishwesh@iitbhilai.ac.in



Recap

- Manual parallelization techniques
- Automatically extracting parallelism
 - Compile time parallelization techniques:
Dependency analysis, dependency testing,
fine-grained parallelism, coarse-grained parallelism
 - Runtime parallelization techniques

Next Two Lectures

- Manual parallelization techniques
- Automatically extracting parallelism
 - Compile time parallelization techniques:
Dependency analysis, dependency testing,
fine-grained parallelism, coarse-grained parallelism
 - Runtime parallelization techniques

Loops with Irregular Memory Access Patterns

```
for (ihop=1; ihop<=nHops; ihop++){  
    Update Subscripts: B1(*) and B2(*)  
    for (itime=1; i<=nTimes; itime++) {  
        for(ih=n; ih<=nParticles; ih++){  
            i = B1[ih];
```

```
for (i=1; i<=n; i++) {  
    for (j=iaL[i]; j<=iaL[i+1]-1; j++)  
{  
    x[i] = x[i] - aL[j] + x[jaL[j]];  
    }  
}
```

Sparse Matrix Computation

```
}  
}  
}
```

Molecular Dynamics

Approaches for Parallelization

```
for i=1 to n
  A[K[i]] = ...;
  /* Some code */
  .. = A[L[i]] op c ;
endfor
```

Approaches:

- Thread Level Speculation
- Inspector-Executor

Approaches for Parallelization

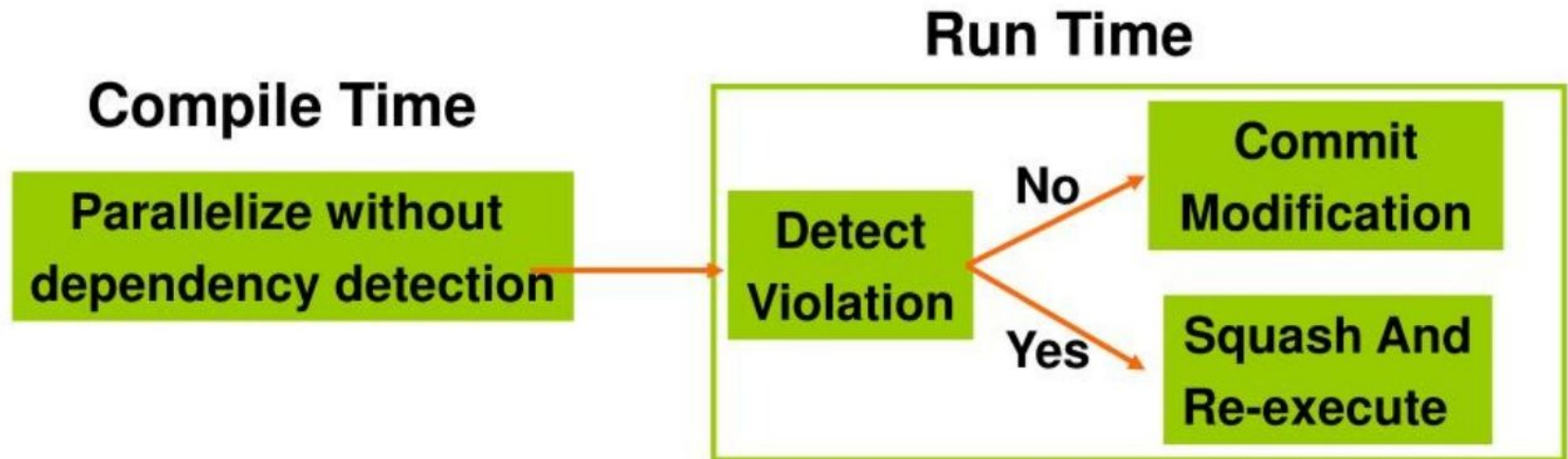
```
for i=1 to n
  A[K[i]] = ...;
  /* Some code */
  .. = A[L[i]] op c ;
endfor
```

Approaches:

- Thread Level Speculation
- Inspector-Executor

Thread Level Speculation (Overview)

- Compilers parallelize the code despite dependencies
- The program is executed in parallel optimistically
- Detect conflicts, recover if found.



Thread Level Speculation

```
for i=1 to n
  A[K[i]] = ...;
  /* Some code */
  .. = A[L[i]] op c ;
endfor
```

- Code can't be parallelized through the compile time parallelization techniques
- Assume they are independent and parallelize
- If conflict is detected, recover it.

Thread Level Speculation

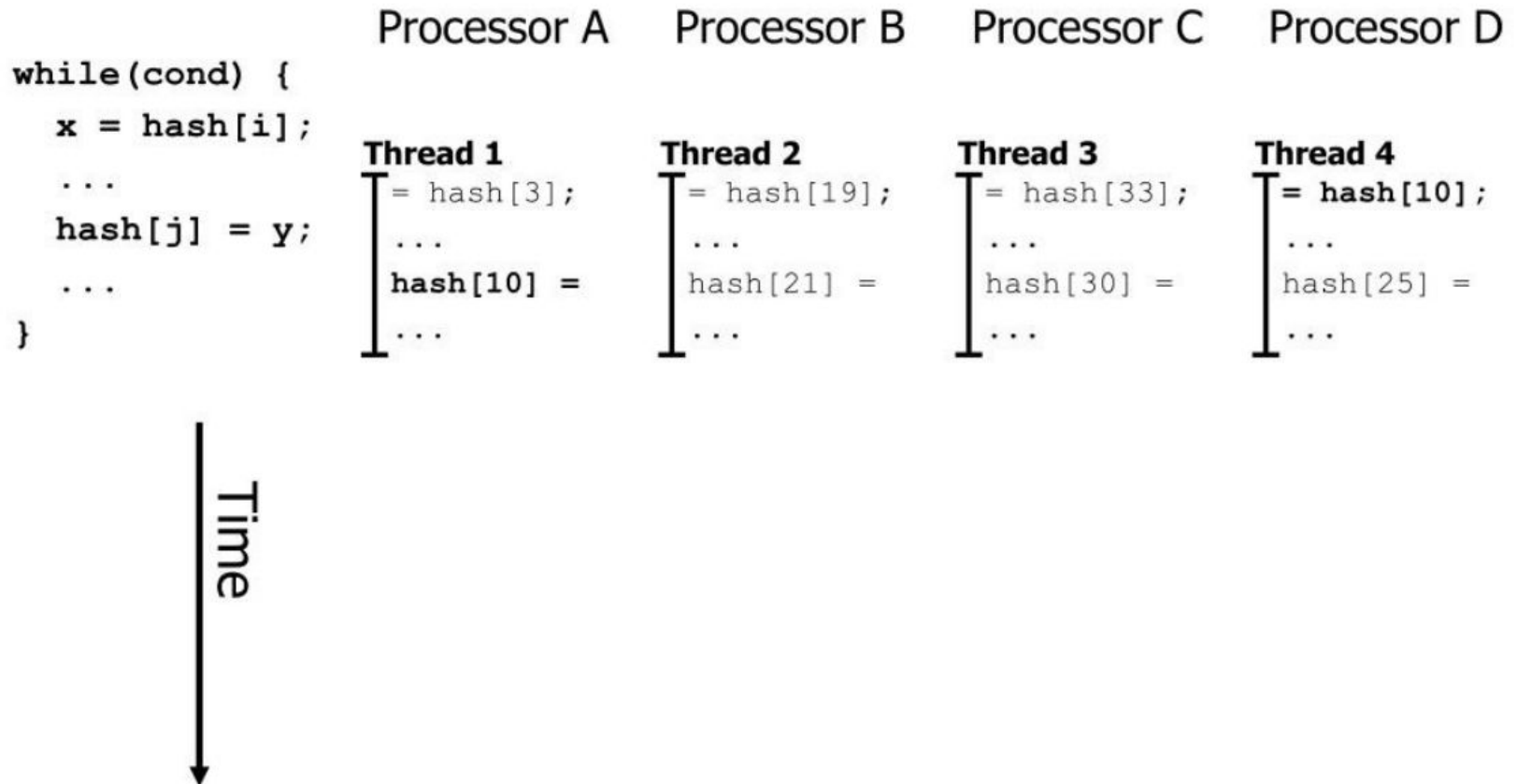
```
while(cond) {  
    x = hash[i];  
    ...  
    hash[j] = y;  
    ...  
}
```

Time
↓

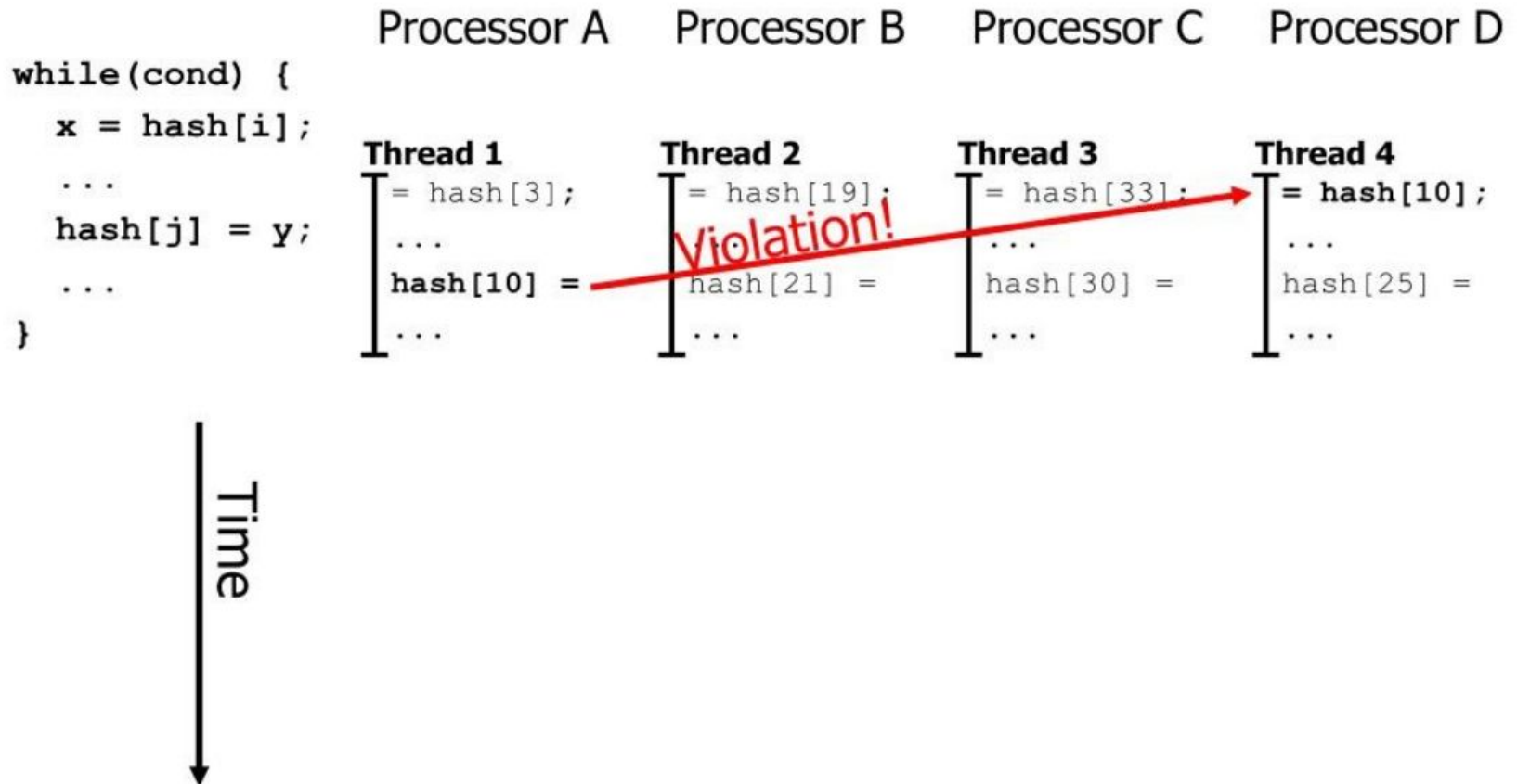
```
| = hash[3];  
| ...  
| hash[10] =  
| ...  
| = hash[19];  
| ...  
| hash[21] =  
| ...  
| = hash[33];  
| ...  
| hash[30] =  
| ...  
| = hash[10];  
| ...  
| hash[25] =  
| ...
```



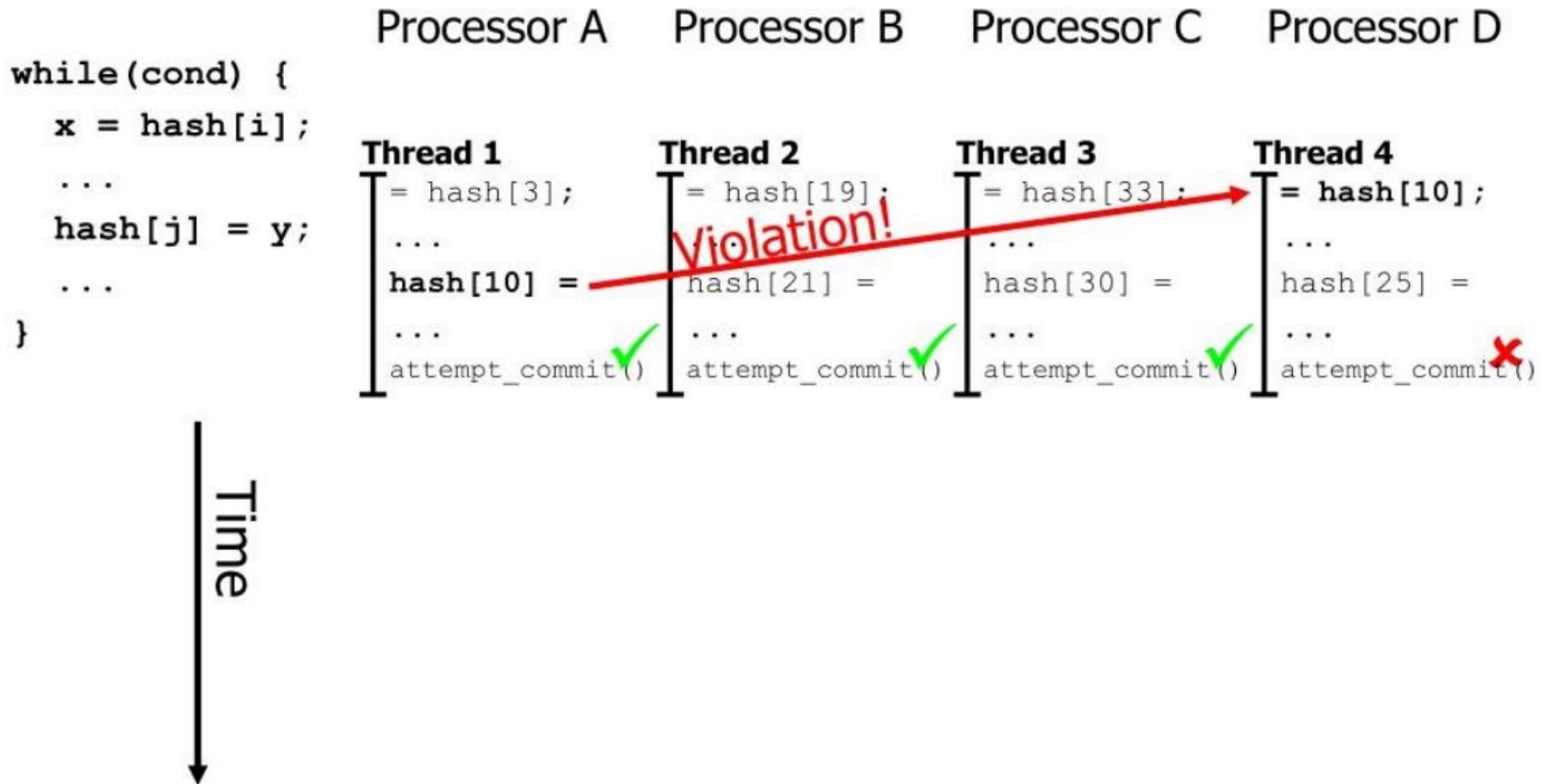
Thread Level Speculation (Example)



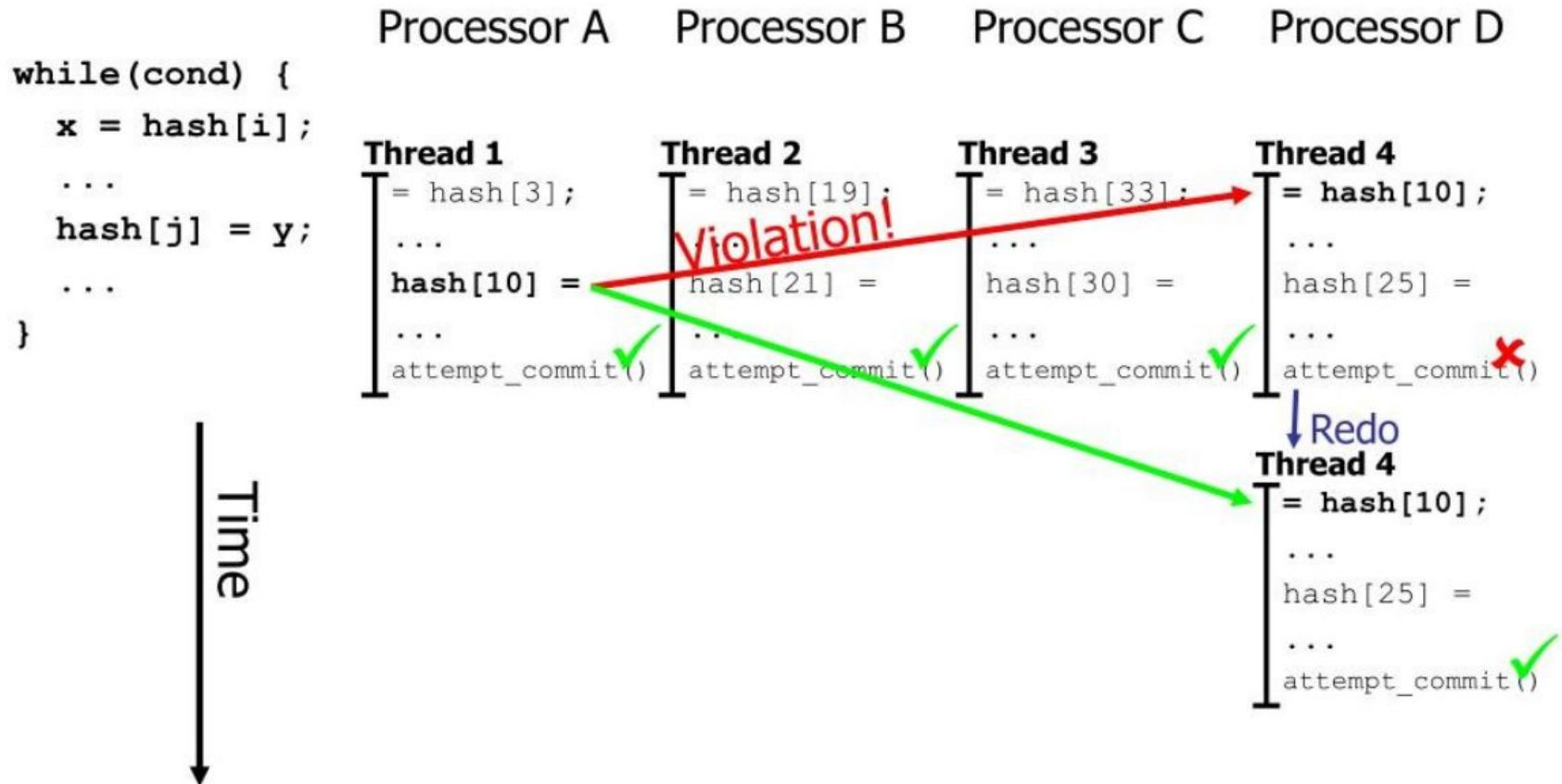
Thread Level Speculation (Example)



Thread Level Speculation (Example)



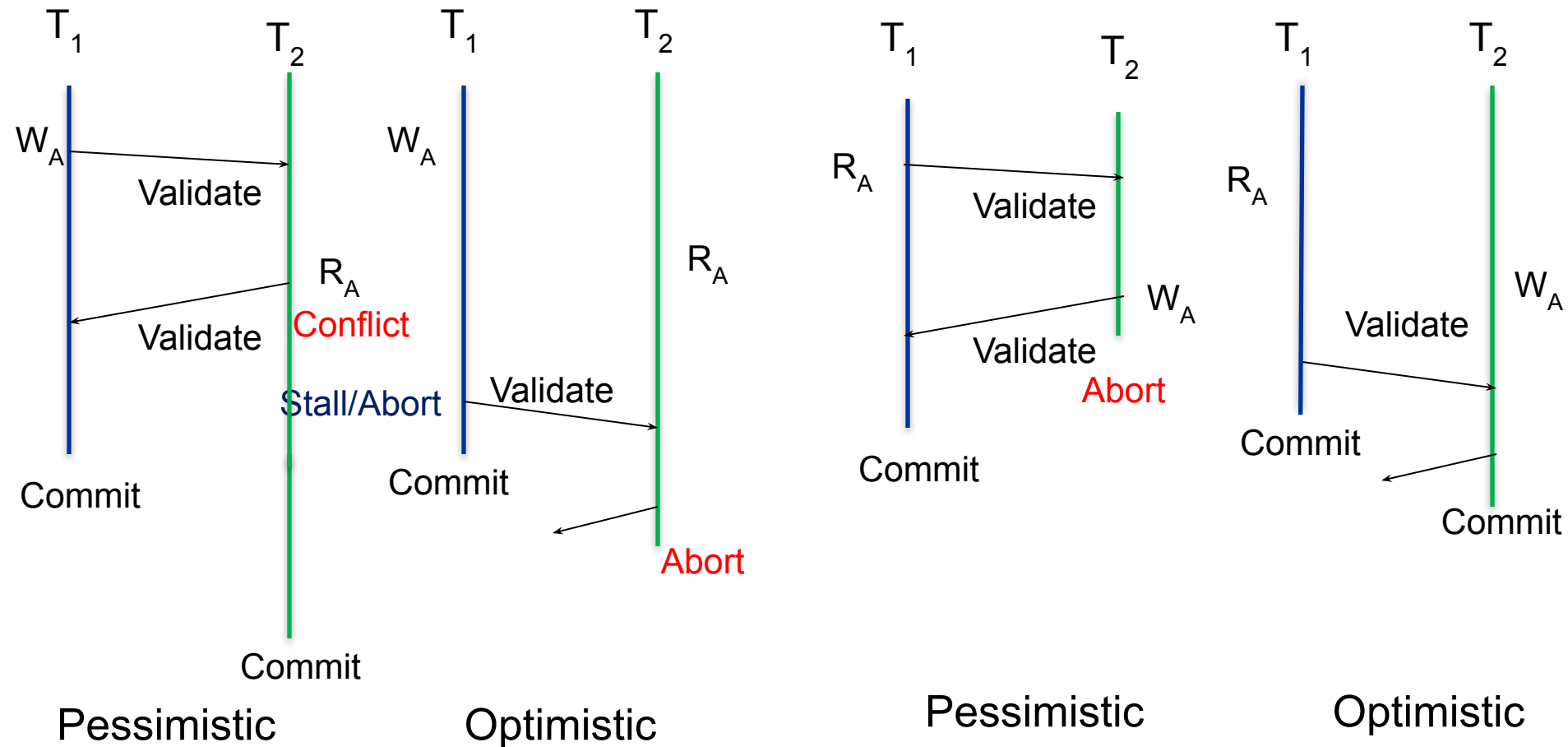
Thread Level Speculation (Example)



Conflict Detection

- Pessimistic
- Optimistic

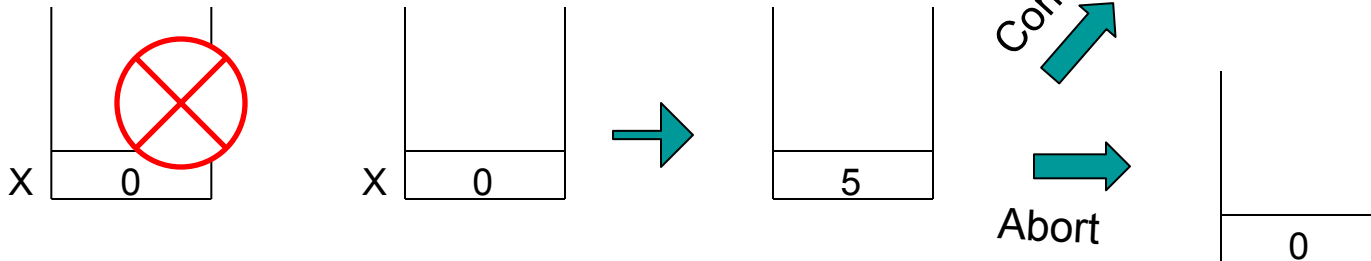
Conflict Detection and Resolution



Version Management

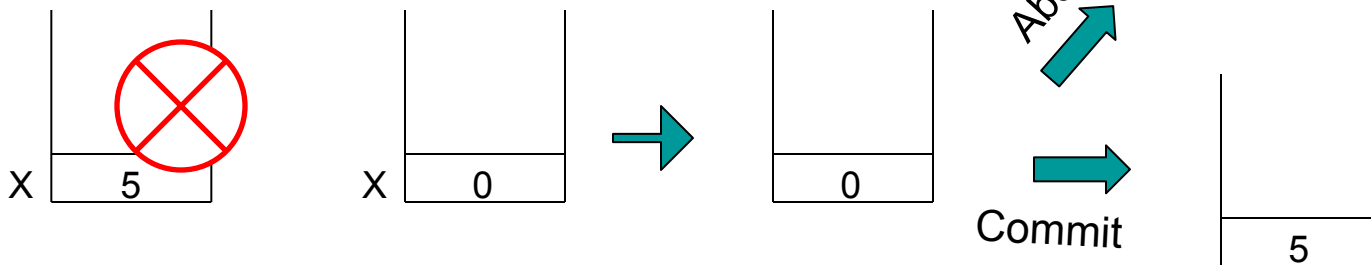
T1: $X = X + 5$

Buffer



Eager Version Management

Buffer



Lazy Version Management

Goals of Speculative Execution

- Extract maximum amount of parallelism
- Efficient conflict detection
- Efficient buffer management
- Minimize performance overheads
- Minimize space overheads

Inspector-Executor

Approaches for Runtime Parallelization

```
for i=1 to n
  A[K[i]] = ...;
  /* Some code */
  .. = A[L[i]] op c ;
endfor
```

Approaches:

- Thread Level Speculation
- Inspector-Executor

Inspector Executor

```
for i=1 to n  
  A[K[i]] = ...;  
  /* Some code */  
  .. = A[L[i]] op c ;  
endfor
```



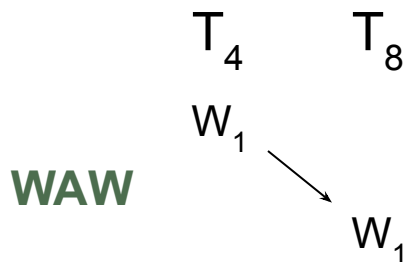
```
Schdle = Inspect__loop();  
Execute_loop(Schdle);
```

- Inspector Phase : Computes the dependencies at runtime in the loop, and prepares a schedule
- Executor Phase: Execute the iterations of for loop according the schedule

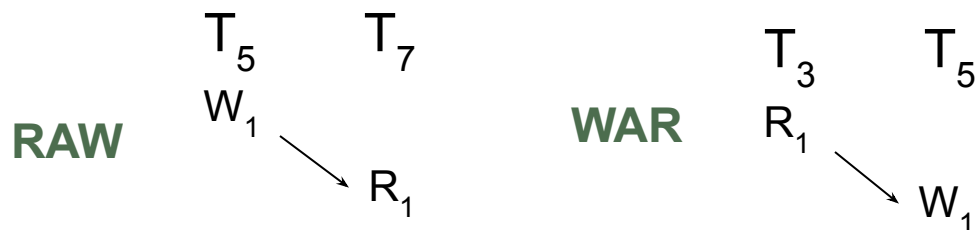
Runtime Dependency Checking

- Each iteration maintains its read and write sets. Also, it maintains a level
- Initially all iterations belong to level 1

Writer Phase (WAW Dependency)



Reader Phase (RAW, WAR Dependencies)



Level Vector

1	2	3	4	5	6	7	8	9
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	2	1
1	1	1	1	1	1	2	2	1
1	1	1	1	1	1	2	2	1

Need to make several passes until all the dependencies are resolved

Runtime Dependency Checking (contd..)

Output after dependency checking:

1	1	2	2	2	1	2	2	3	3	1	2	2	3	4	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

```
Procedure ExecPhase
    level = 1
    while (level <= total_levels)
        KernelExec(..)
        level = level+1
    end while
end procedure
```

Execution Phase

Analysis

- Can extract parallelism at runtime
- Additional computational overhead for inspection if no parallelism
- Additional space overhead
 - Keep track of dependencies



Thank you!